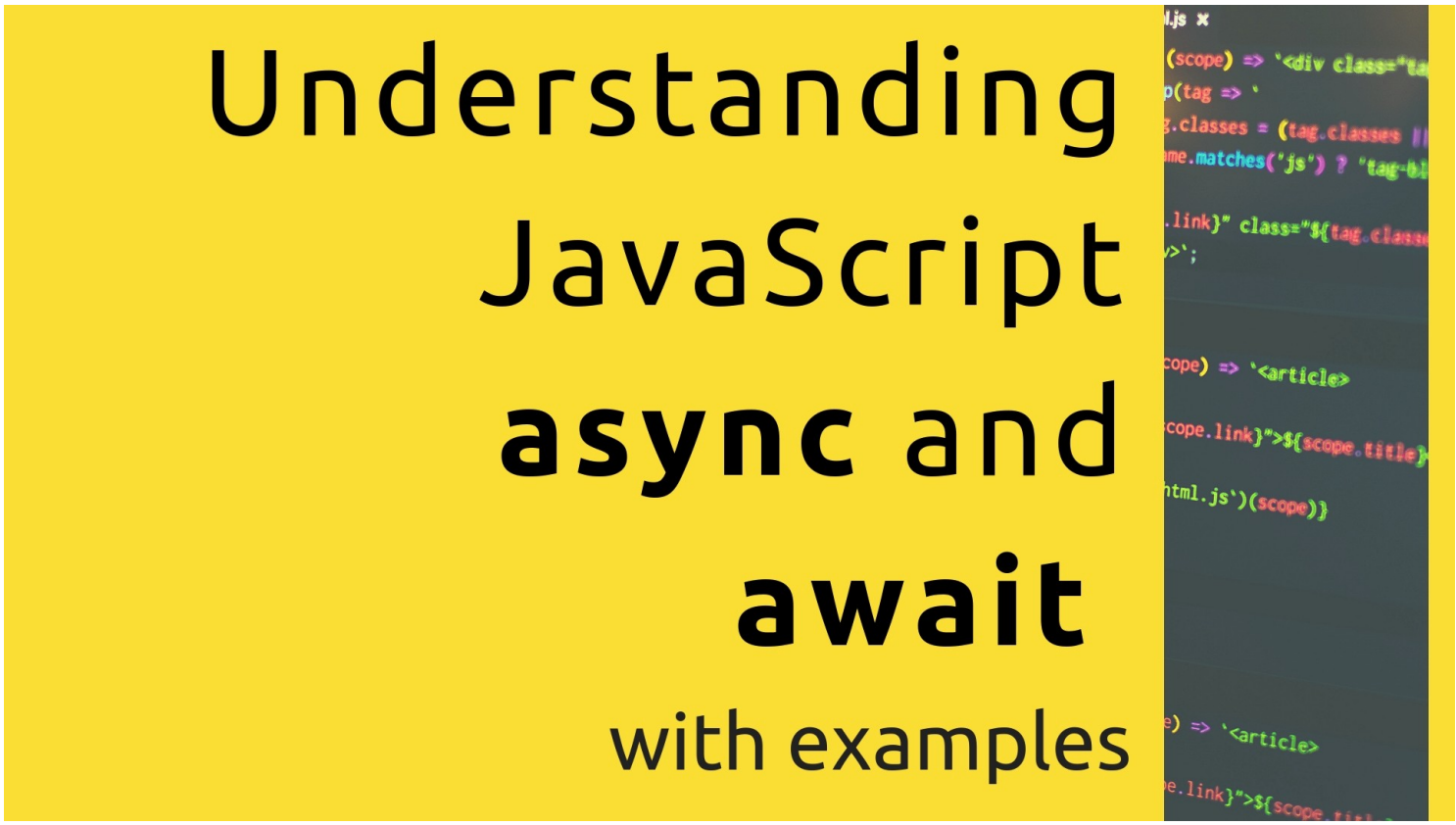


# Deeply Understanding JavaScript Async and Await with Examples

Arfat Salman Follow
May 1, 2019 · 10 min read

In the beginning, there were callbacks. **A callback is nothing special but a function that is executed at some later time.** Due to JavaScript's asynchronous nature, a callback is required in many places, where the results are not available immediately.

Here's an example of reading a file in Node.js (asynchronously) —

```
fs.readFile(__filename, 'utf-8', (err, data) => {
  if (err) {
    throw err;
  }
  console.log(data);
});
```

Problems arise when we want to do multiple async operations. Imagine this hypothetical scenario (where all operations are async) —

- We query our database for the user `Arfat`. We read the `profile_img_url` and fetch the image from `someServer.com`.
- After fetching the image, we transform it into a different format, say PNG to JPEG.
- If the transformation is successful, we send the user an email.
- We log this task in our file `transformations.log` with the timestamp.

The code for something like this would like —

```
queryDatabase({ username: 'Arfat'}, (err, user) => {
  // handle errors database querying failure
  const image_url = user.profile_img_url;
  getImageByURL(`someServer.com/q=${image_url}`, (err, image) => {
    // handle errors fetching failure
    transformImage(image, (err, transformedImage) => {
      // handle errors transformation failure
      sendEmail(user.email, (err) => {
        // handle errors of email failure
        logTaskInFile('transformed the file and sent user an email', (err) => {
          // handle errors of logging failure
        })
      })
    })
  })
})
```

Example of Callback hell.

**Note the nesting of callbacks and the staircase of `})` at the end.** This is affectionately called as **Callback Hell** or **Pyramid of Doom** due to its namesake resemblance. Some disadvantages of this are —

- The code becomes harder to read as one has to move from left to right to understand.
- Error handling is complicated and often leads to bad code.

To overcome this problem, JavaScript gods created **Promises**. **Now, instead of nesting callbacks inward, we can chain them.** Here's an example —

```
queryDatabase({ username: 'Arfat'})
  .then((user) => {
    const image_url = user.profile_img_url;
    return getImageByURL(`someServer.com/q=${image_url}`)
      .then(image => transformImage(image))
      .then(() => sendEmail(user.email))
  })
  .then(() => logTaskInFile('...'))
  .catch(() => handleErrors()) // handle all errors
```

Using Promises

The flow has become a familiar *top-to-bottom* rather than *left-to-right* as in callbacks, which is a plus. However, Promises still suffer from some problems —

- We still have to give a callback to every `.then`.
- Instead of using a normal `try/catch`, we have to use `.catch` for error handling.
- **Looping over multiple promises in a sequence is challenging and non-intuitive.**

As a demonstration of the last point, **try this challenge**!

## The Challenge

*Let's assume that we have a `for` loop that prints 0 to 10 at random intervals (0 to n seconds). We need to modify it using promises to print sequentially 0 to 10. For example, if 0*

*takes 6 seconds to print and 1 takes two seconds to print, then 1 should wait for 0 to print and so on.*

Needless to say, don't use `async/await` or `.sort` function. We'll have a solution towards the end.

. . .

Tip: **Build more modular JavaScript software**. Use tools like **Bit** (GitHub) to develop, share and sync modular components between your projects. Modular software is faster to build, easier to maintain and simpler to reuse. Try it:



. . .

## Async Functions

Introduced in **ES2017**(ES8), async functions make working with promises much easier.

- **It is important to note the async functions work on top of promises.**
- They are not a fundamentally different concept.
- They can be thought of as an alternate way of writing promise-based code.
- We can **avoid chaining promise** altogether using async/await.
- They allow asynchronous execution while **maintaining a regular, *synchronous* feel**.

Hence, an **understanding of promises is required** before you can fully understand async/await concepts.

## Syntax

They consist of two main keywords- async and await. **`async` is used to make a function asynchronous.** It **unlocks** the use of `await` inside these functions. Using `await` in any other case is a syntax error.

```
// With function declaration

async function myFn() {
  // await ...
}

// With arrow function

const myFn = async () => {
  // await ...
}

function myFn() {
  // await fn(); (Syntax Error since no async)
}
```

Notice the use of `async` keyword **at the beginning** of the function declaration. In the case of arrow function, `async` is put after the `=` sign and before the parentheses.

Async functions can also be put on an object as methods, or in class declarations as follows.

```
// As an object's method

const obj = {
  async getName() {
    return fetch('https://www.example.com');
  }
}

// In a class
```

```
class Obj {
  async getResource() {
    return fetch('https://www.example.com');
  }
}
```

**Note: Class constructors and getters/setters *cannot* be async.**

## Semantics and Evaluation Rules

Async functions are normal JavaScript functions with the following differences —

### An async function always returns a promise.

```
async function fn() {
  return 'hello';
}

fn().then(console.log)
// hello
```

The function `fn` returns `'hello'`. Because we have used `async`, the return value `'hello'` is **wrapped in a promise** (via `Promise` constructor).

Here's an **alternate representation** without using `async` —

```
function fn() {
  return Promise.resolve('hello');
}

fn().then(console.log);
// hello
```

In this, we are **manually returning a promise** instead of using `async`.

A slightly more accurate way to say the same thing — **The body of an async function is always wrapped in a new `Promise`.**

If the return value is primitive, `async` functions return a **promise-wrapped version of the value.** However, when the return value is a promise object, **its resolution is returned in a new promise.** *[See this comment]*

```
// in case of primitive values

const p = Promise.resolve('hello')
p instanceof Promise;
// true

// p is returned as is it

Promise.resolve(p) === p;
// true
```

### What happens when you *throw an error* inside an **async** function?

For example —

```
async function foo() {
  throw Error('bar');
}

foo().catch(console.log);
```

`foo()` will return a ***rejected*** promise if the error is **uncaught**. Instead of `Promise.resolve`, `Promise.reject` wraps the error and is returned. See E*rror Handling* section later.

The net effect is that you return whatever you want, **you will always get a promise out of an async function**.

## Async functions pause at each `await` <expression>.

An `await` acts on an *expression*. When the expression is a promise, **the evaluation of the async function halts until the promise is resolved.** When the expression is a non-promise value, it is converted to a promise using `Promise.resolve` and then resolved.

```
// utility function to cause delay
// and get random value

const delayAndGetRandom = (ms) => {
  return new Promise(resolve => setTimeout(
    () => {
      const val = Math.trunc(Math.random() * 100);
      resolve(val);
    }, ms
  ));
};

async function fn() {
  const a = await 9;
  const b = await delayAndGetRandom(1000);
  const c = await 5;
  await delayAndGetRandom(1000);

  return a + b * c;
}

// Execute fn
fn().then(console.log);
```

Let's examine the function `fn` line by line —

- When `fn` is executed, the first line to be evaluated is `const a = await 9;`. It is **internally transformed** into `const a = await Promise.resolve(9);`.
- Since we are using `await`, `fn` **pauses until the variable** `a` gets a value. In this case, the **promise resolves it to** `9`.
- `delayAndGetRandom(1000)` causes `fn` to pause until the function `delayAndGetRandom` is resolved which is after 1 second. So, `fn` effectively pauses for 1 second.
- Also, `delayAndGetRandom` resolves with a random value. Whatever is passed in the `resolve` function, that is assigned to the variable `b`.
- `c` gets the value of `5` similarly and we delay for 1 second again using `await delayAndGetRandom(1000)`. We don't use the resolved value in this case.

- Finally, we compute the result `a + b * c` which is wrapped in a Promise using `Promise.resolve`. This wrapped promise is returned.

**Note:** If this pause and resume are reminding you of ES6 **generators**, it's because there are **good reasons** for it.

## The Solution

Let's use async/await for the hypothetical problem listed at the beginning of the article —

Using async/await

We make an async function `finishMyTask` and use `await` to wait for the result of operations such as `queryDatabase`, `sendEmail`, `logTaskInFile` etc.

If we contrast this solution with the solutions using promises above, we find that **it is roughly the same line of code**. However, async/await has made it simpler in terms of syntactical complexity. **There aren't multiple callbacks and `.then` / `.catch` to remember.**

Now, let's solve the **challenge of numbers** listed above. Here are two implementations —

"Challenge" Implementations

If you want, you can run them yourself at *repl.it* **console**.

If you were allowed **async** function, the task would have been much simpler.

```
async function printNumbersUsingAsync() {
  for (let i = 0; i < 10; i++) {
    await wait(i, Math.random() * 1000);
    console.log(i);
  }
}
```

This implementation is also provided in the *repl.it* **console**.

## Error Handling

As we saw in the **Semantics** section, an **uncaught** `Error()` is wrapped in a rejected promise. However, we can use `try-catch` in async functions to **handle errors synchronously**. Let's begin with this utility function —

```
async function canRejectOrReturn() {
  // wait one second
  await new Promise(res => setTimeout(res, 1000));

// Reject with ~50% probability
  if (Math.random() > 0.5) {
    throw new Error('Sorry, number too big.')
  }

return 'perfect number';
}
```

`canRejectOrReturn()` is an async function and it will either **resolve with** `'perfect number'` or **reject with** `Error('Sorry, number too big')`.

Let's look at the code example —

```
async function foo() {
  try {
    await canRejectOrReturn();
  } catch (e) {
    return 'error caught';
  }
}
```

Since we are awaiting `canRejectOrReturn`, **its own rejection will be turned into a throw** and the `catch` block will execute. That is, `foo` will **either resolve with** `undefined` (because we are not returning anything in `try`) or **it will resolve with** `'error caught'`. It will never reject since we used a `try-catch` block to handle the error in the `foo` function itself.

Here's another example —

```
async function foo() {
  try {
    return canRejectOrReturn();
  } catch (e) {
    return 'error caught';
  }
}
```

Note that **we are returning** (and not awaiting) `canRejectOrReturn` from `foo` this time. `foo` will either **resolve with** `'perfect number'` or **reject with** `Error('Sorry, number too big')`. **The** `catch` **block will never be executed.**

It is because we **return the promise returned** by `canRejectOrReturn`. Hence, the resolution of `foo` becomes the resolution of `canRejectOrReturn`. You can break `return canRejectOrReturn()` into two lines to see clearly (**Note the missing await** in the first line)—

```
try {
    const promise = canRejectOrReturn();
    return promise;
}
```

Let's see the usage of `await` and `return` together —

```
async function foo() {
  try {
    return await canRejectOrReturn();
  } catch (e) {
    return 'error caught';
  }
}
```

In this case, `foo` **resolves with either** `'perfect number'` or **resolve with** `'error caught'`. **There is no rejection.** It is like the first example above with just `await`. Except, we **resolve with the value that** `canRejectOrReturn` produces rather than `undefined`.

You can break `return await canRejectOrReturn();` to see the effect —

```
try {
    const value  = await canRejectOrReturn();
    return value;
}
// ...
```

## Common Mistakes and Gotchas

Because of an intricate play of Promise-based and async/await concepts, there are some subtle errors that can creep into the code. Let's look at them —

### Not using await

In some cases, **we forget to use** the `await` keyword before a promise or return it. Here's an example —

```
async function foo() {
  try {
    canRejectOrReturn();
  } catch (e) {
    return 'caught';
  }
}
```

Note that we are not using `await` or `return`. `foo` **will always resolve with** `undefined` **without waiting for 1 second**. However, the promise **does start** its execution. If there are side-effects, **they will happen**. If it throws an error or rejects, then `UnhandledPromiseRejectionWarning` will be issued.

### async functions in callbacks

We often use async functions in `.map` or `.filter` as callbacks. Let's take an example — Suppose we have a function `fetchPublicReposCount(username)` that fetched the number of public GitHub repositories a user has. We have three users whose counts we want to fetch. Let's see the code —

```
const url = 'https://api.github.com/users';

// Utility fn to fetch repo counts
const fetchPublicReposCount = async (username) => {
  const response = await fetch(`${url}/${username}`);
  const json = await response.json();
  return json['public_repos'];
}
```

We want to fetch repo counts of `['ArfatSalman', 'octocat', 'norvig']`. We may do something like this —

```
const users = [
  'ArfatSalman',
  'octocat',
  'norvig'
];

const counts = users.map(async username => {
  const count = await fetchPublicReposCount(username);
  return count;
});
```

Note the `async` in the callback to the `.map`. We might expect that `counts` variable will contain the numbers of repos. However, as we have seen earlier, **all async functions return promises.** Hence, `counts` is actually **an array of promises.** `.map` calls the anonymous callback with every `username`, and a promise is returned with every invocation which `.map` keeps in the resulting array.

## Too sequential using await

We may also think of a solution such as —

```
async function fetchAllCounts(users) {
  const counts = [];
  for (let i = 0; i < users.length; i++) {
    const username = users[i];
    const count = await fetchPublicReposCount(username);
    counts.push(count);
  }
```

```
  return counts;
}
```

We are manually fetching each count, and appending them in the `counts` array. The **problem with this code** is that until the first username's count is fetched, **the next will not start.** At a time, **only one repo count** is fetched.

If a single fetch takes 300 ms, then `fetchAllCounts` will take ~900ms for 3 users. As we can see that time usage will linearly grow with user counts. Since the **fetching of repo counts is not co-dependent**, we can **parallelize the operation.**

We can fetch users concurrently instead of doing them sequentially. We are going to utilize `.map` and `Promise.all`.

```
async function fetchAllCounts(users) {
  const promises = users.map(async username => {
    const count = await fetchPublicReposCount(username);
    return count;
  });
  return Promise.all(promises);
}
```

`Promise.all` receives an array of promises as input and returns a promise as output. The returned promise resolves with **an array of all promise resolutions or rejects with the first rejection.** However, it may not be feasible to start all promises at the same time. Maybe you want to complete promises in batches. You can look at **p-map** for limited concurrency.

## Conclusion

Async functions have become really important. With the introduction of Async Iterators, async functions will see more adoption. It is important to have a good understanding of them for modern JavaScript developer. I hope this article sheds some light on that. :)

. . .